

工作坊：無伺服器網路 API

來源：<https://codelabs.developers.google.com/codelabs/serverless-web-apis?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將以 Google Cloud 無伺服器解決方案為基礎，開發一個 Web API，以提供書架和相關書籍。您將建立一個 Cloud 函式來匯入範例資料、建立 Cloud Run 容器以提供可重複使用的後端 Web API，以及提供網路前端來提供瀏覽書籍庫的 App Engine 網頁應用程式。

目錄

1. 總覽
2. 設定和需求
3. 準備環境並啟用 Cloud API
4. 取得程式碼
5. 範例書籍程式庫資料
6. 匯入範例書籍資料的函式端點
7. REST API 合約
8. 在容器中部署及公開 REST API
9. 託管網頁應用程式，瀏覽程式庫
10. 清理 (選用)
11. 恭喜！

步驟 1 · 約 3 分鐘

1. 總覽

本程式碼研究室的目標是體驗 Google Cloud Platform 提供的「無伺服器」服務：

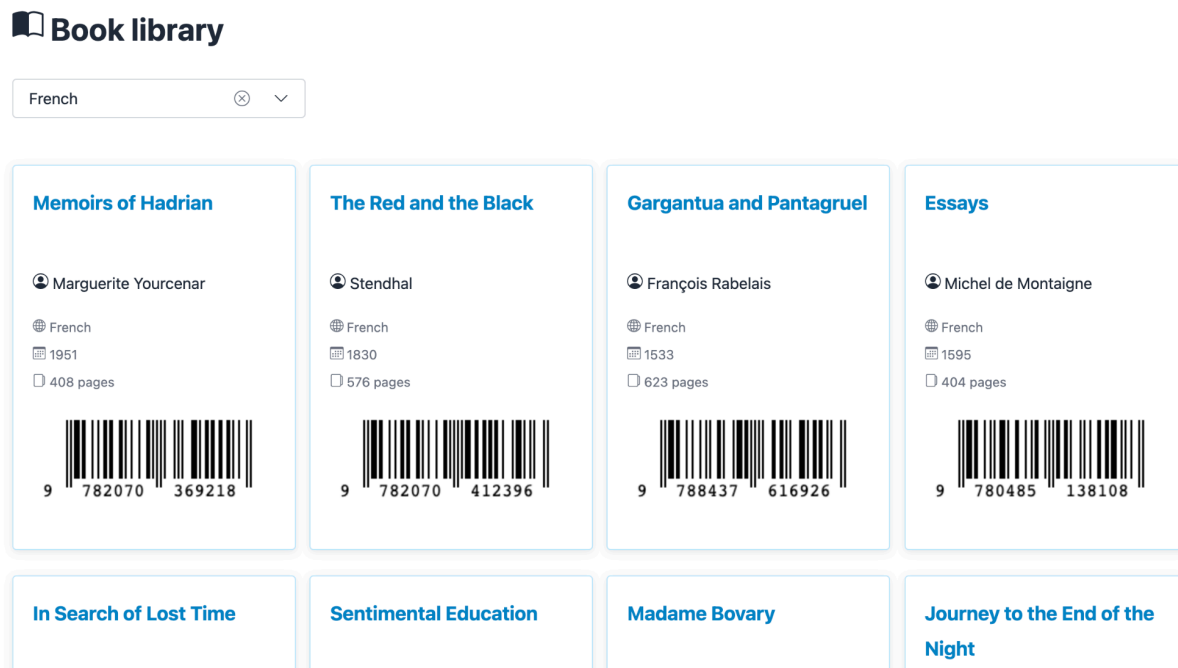
- [Cloud Functions](#)：以函式形式部署小型商業邏輯單元，對各種事件 (Pub/Sub 訊息、Cloud Storage 中的新檔案、HTTP 要求等) 做出反應。
- [App Engine](#)：用於部署及提供網頁應用程式、網頁 API、行動後端和靜態資產，並具備快速擴充及縮減功能。
- [Cloud Run](#)：部署及擴充容器，其中可包含任何語言、執行階段或程式庫。

並瞭解如何運用這些無伺服器服務部署及擴充 Web 和 REST API，同時瞭解一些良好的 RESTful 設計原則。

在本研討會中，我們將建立書架探索器，其中包含：

- Cloud 函式：將圖書館提供的書籍初始資料集匯入 [Cloud Firestore](#) 文件資料庫。
- Cloud Run 容器：透過資料庫內容公開 REST API，
- App Engine 網頁前端：呼叫 REST API，瀏覽書籍清單。

完成本程式碼研究室後，網路前端會如下所示：



課程內容

- Cloud Functions
 - Cloud Firestore
 - Cloud Run
 - App Engine
-

步驟 2 · 約 2 分鐘

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots from the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu in the top navigation bar, with a blue arrow pointing to it. The bottom screenshot shows the 'New Project' form. The 'Project name' field contains 'My Project 13475'. The 'Project ID' field contains 'inbound-analogy-389614', which is highlighted with a blue box. The 'Location' field is empty, and the 'BROWSE' button is visible. The 'CREATE' and 'CANCEL' buttons are at the bottom.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

用者可參加[價值\\$300 美元的免費試用計畫](#)。

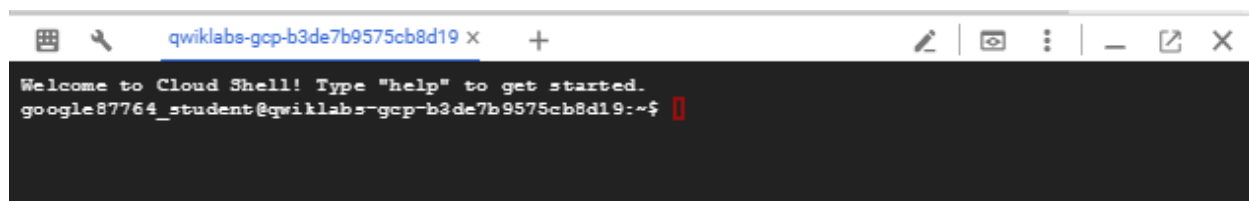
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Google Cloud Shell interface. The browser's address bar displays 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal window has a black background with white text. The text reads: 'Welcome to Cloud Shell! Type "help" to get started.' followed by a new line with the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor. The browser window includes standard navigation buttons and a tab titled 'qwiklabs-gcp-b3de7b9575cb8d19 x'.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

3. 準備環境並啟用 Cloud API

為了使用這個專案中需要的各種服務，我們將啟用幾個 API。我們將在 Cloud Shell 中啟動下列指令，藉此完成這項操作：

```
$ gcloud services enable \  
  appengine.googleapis.com \  
  cloudbuild.googleapis.com \  
  cloudfunctions.googleapis.com \  
  compute.googleapis.com \  
  firestore.googleapis.com \  
  run.googleapis.com
```

過一段時間後，您應該會看到作業順利完成：

```
Operation "operations/acf.5c5ef4f6-f734-455d-b2f0-ee70b5a17322" finished  
successfully.
```

我們也會設定環境變數，以供後續使用：部署函式、應用程式和容器的雲端區域：

```
$ export REGION=europe-west3
```

注意：並非所有 Google Cloud Platform 產品都適用於所有區域！在本程式碼研究室中，Cloud Functions、App Engine、Cloud Run 和 Cloud Firestore 都可在 `europe-west3` 中使用。

如要進一步瞭解各項產品的供應情形，請參閱[服務據點說明文件](#)頁面。

由於我們會在 Cloud Firestore 資料庫中儲存資料，因此需要建立資料庫：

```
$ gcloud app create --region=${REGION}  
$ gcloud firestore databases create --location=${REGION}
```

注意：為了建立資料庫，Google Cloud Platform 實際上需要有可用的 App Engine 專案，即使我們尚未在該平台上部署應用程式也一樣。

在本程式碼研究室的後續部分實作 REST API 時，我們需要排序及篩選資料。為此，我們將建立三個索引：

```
$ gcloud firestore indexes composite create --collection-group=books \  
  --field-config field-path=language,order=ascending \  
  --field-config field-path=updated,order=descending  
  
$ gcloud firestore indexes composite create --collection-group=books \  
  --field-config field-path=author,order=ascending \  
  --field-config field-path=updated,order=descending
```

注意：建立索引作業可能相當耗時，每次作業需要 1 分鐘到 10 分鐘不等，因為這類作業會在 Google Cloud Platform 上分批完成。因此請耐心等待，或在背景執行這些指令，或在其他 Cloud Shell 終端機中執行。

這 3 個索引分別對應我們將依作者或語言執行的搜尋，同時透過更新的欄位維持集合中的排序。

步驟 4 · 約 2 分鐘

4. 取得程式碼

從下列 GitHub 存放區取得程式碼：

```
$ git clone https://github.com/glaforge/serverless-web-apis
```

應用程式程式碼是以 [Node.JS](#) 編寫。

您會看到下列資料夾結構，與本實驗室相關：

```
serverless-web-apis
|
|—— data
|   |—— books.json
|
|—— function-import
|   |—— index.js
|   |—— package.json
|
|—— run-crud
|   |—— index.js
|   |—— package.json
|   |—— Dockerfile
|
|—— appengine-frontend
    |—— public
        |—— css/style.css
        |—— html/index.html
        |—— js/app.js
    |—— index.js
    |—— package.json
    |—— app.yaml
```

相關資料夾如下：

- `data`：這個資料夾包含 100 本書籍的範例資料清單。
 - `function-import`：這個函式會提供匯入範例資料的端點。
 - `run-crud` - 這個容器會公開 Web API，以便存取儲存在 Cloud Firestore 中的書籍資料。
 - `appengine-frontend`：這個 App Engine 網頁應用程式會顯示簡單的唯讀前端，方便您瀏覽書籍清單。
-

5. 範例書籍程式庫資料

在資料夾中，我們有一個 `books.json` 檔案，內含一百本書的清單，應該值得一讀。這個 JSON 文件是包含 JSON 物件的陣列。我們來看看透過 Cloud 函式擷取的資料形狀：

```
[
  {
    "isbn": "9780435272463",
    "author": "Chinua Achebe",
    "language": "English",
    "pages": 209,
    "title": "Things Fall Apart",
    "year": 1958
  },
  {
    "isbn": "9781414251196",
    "author": "Hans Christian Andersen",
    "language": "Danish",
    "pages": 784,
    "title": "Fairy tales",
    "year": 1836
  },
  ...
]
```

這個陣列中的所有書籍項目都包含下列資訊：

- `isbn`：識別書籍的 ISBN-13 代碼。
 - `author`：書籍作者的姓名。
 - `language`：書籍撰寫時使用的口語。
 - `pages`：書籍頁數。
 - `title`：書籍名稱。
 - `year`：書籍出版年份。
-

步驟 6 · 約 7 分鐘

6. 匯入範例書籍資料的函式端點

在第一個部分中，我們會實作用於匯入範例書籍資料的端點。我們會使用 Cloud Functions 達成此目的。

探索程式碼

首先，請查看 `package.json` 檔案：

```
{
  "name": "function-import",
  "description": "Import sample book data",
  "license": "Apache-2.0",
  "dependencies": {
    "@google-cloud/firestore": "^4.9.9"
  },
  "devDependencies": {
    "@google-cloud/functions-framework": "^3.1.0"
  },
  "scripts": {
    "start": "npx @google-cloud/functions-framework --target=parseBooks"
  }
}
```

在執行階段依附元件中，我們只需要 `@google-cloud/firestore` NPM 模組來存取資料庫及儲存書籍資料。在幕後，Cloud Functions 執行階段也會提供 Express 網頁架構，因此我們不需要將其宣告為依附元件。

在開發依附元件中，我們宣告了 `Functions Framework` (`@google-cloud/functions-framework`)，這是用於叫用函式的執行階段架構。這項開放原始碼框架也能在本機 (在本例中為 Cloud Shell 內) 使用，不必在每次變更時都部署函式，因此可改善開發回饋循環。

若要安裝依附元件，請使用 `install` 指令：

```
$ npm install
```

`start` 指令碼會使用 Functions Framework，提供您可透過下列指令在本機執行的函式指令：

```
$ npm start
```

您可以使用 `curl` 或 Cloud Shell 網頁預覽功能 (如果適用)，透過 HTTP GET 要求與函式互動。

現在來看看 `index.js` 檔案，其中包含書籍資料匯入函式的邏輯：

```
const Firestore = require('@google-cloud/firestore');
const firestore = new Firestore();
const bookStore = firestore.collection('books');
```

我們將 Firestore 模組例項化，並指向書籍集合 (類似於關聯式資料庫中的表格)。

```

functions.http('parseBooks', async (req, resp) => {
  if (req.method !== "POST") {
    resp.status(405).send({error: "Only method POST allowed"});
    return;
  }
  if (req.headers['content-type'] !== "application/json") {
    resp.status(406).send({error: "Only application/json accepted"});
    return;
  }
  ...
})

```

我們正在匯出 `parseBooks` JavaScript 函式。這是我們稍後部署時會宣告的函式。

接下來的幾項指令會檢查下列事項：

- 我們只接受 HTTP `POST` 要求，否則會傳回 `405` 狀態碼，表示不允許其他 HTTP 方法。
- 我們只接受 `application/json` 酬載，否則會傳送 `406` 狀態碼，指出酬載格式不符規定。

```

const books = req.body;

const writeBatch = firestore.batch();

for (const book of books) {
  const doc = bookStore.doc(book.isbn);
  writeBatch.set(doc, {
    title: book.title,
    author: book.author,
    language: book.language,
    pages: book.pages,
    year: book.year,
    updated: Firestore.Timestamp.now()
  });
}

```

接著，我們就能透過要求的 `body` 擷取 JSON 酬載。我們正在準備 Firestore 批次作業，以便大量儲存所有書籍。我們會疊代由書籍詳細資料組成的 JSON 陣列，並逐一檢查 `isbn`、`title`、`author`、`language`、`pages` 和 `year` 欄位。書籍的 ISBN 碼會做為其主鍵或識別碼。

```

try {
  await writeBatch.commit();
  console.log("Saved books in Firestore");
} catch (e) {
  console.error("Error saving books:", e);
  resp.status(400).send({error: "Error saving books"});
  return;
};

resp.status(202).send({status: "OK"});

```

現在大部分的資料都已準備就緒，我們可以提交作業。如果儲存作業失敗，我們會傳回 `400` 狀態碼，表示作業失敗。否則，我們可以傳回 OK 回應，並以 `202` 狀態碼表示系統已接受大量儲存要求。

執行及測試匯入函式

執行程式碼前，請先使用下列指令安裝依附元件：

```
$ npm install
```

如要在本機執行函式，請使用我們在 `package.json` 中定義的 `start` 指令碼指令，這要歸功於 Functions Framework：

```
$ npm start

> start
> npx @google-cloud/functions-framework --target=parseBooks

Serving function...
Function: parseBooks
URL: http://localhost:8080/
```

如要將 HTTP `POST` 要求傳送至本機函式，可以執行：

```
$ curl -d "@../data/books.json" \
  -H "Content-Type: application/json" \
  http://localhost:8080/
```

啟動這項指令時，您會看到下列輸出內容，確認函式在本機執行：

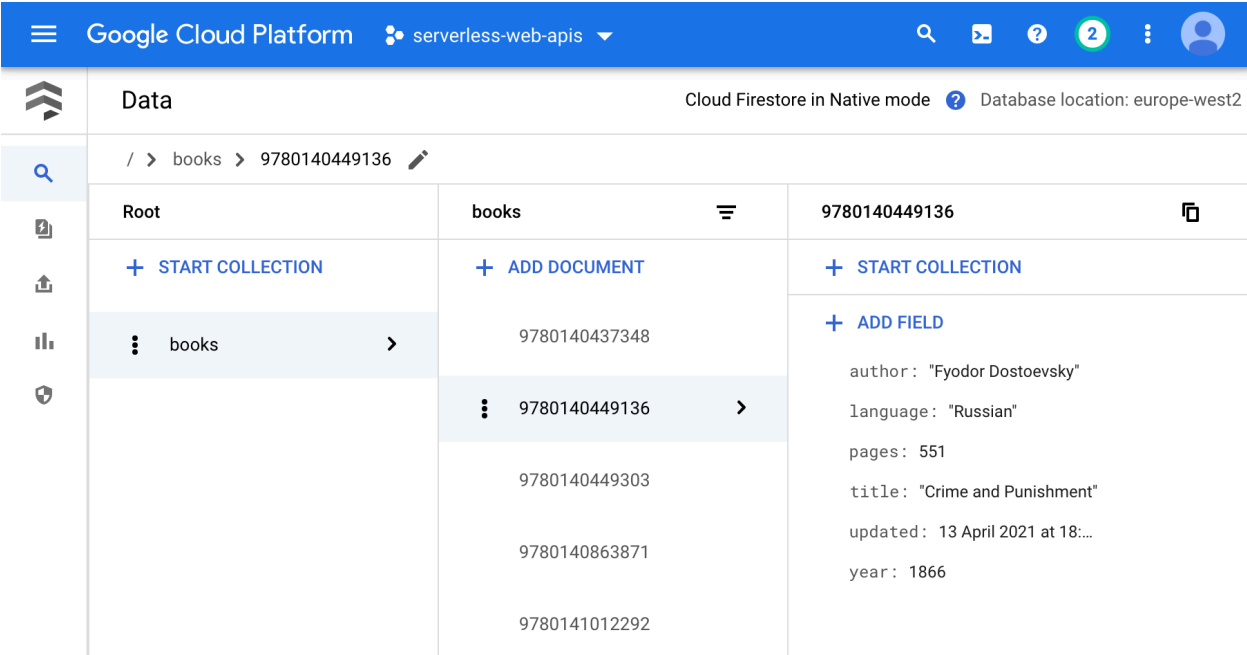
```
{"status": "OK"}
```

注意：如果搭配 `curl` 使用 `-v` 詳細程度旗標，您會看到一些額外資訊，例如回應詳細資料，包括傳回的 HTTP 標頭、傳回的狀態碼：

```
< HTTP/1.1 202 Accepted
< X-Powered-By: Express
< Content-Type: application/json; charset=utf-8
< Content-Length: 15
< ETag: W/"f-v/YlJusChTxrQUzPtNAKycooOTA"
< Date: Tue, 13 Apr 2021 16:41:48 GMT
< Connection: keep-alive
<
* Connection #0 to host localhost left intact

{"status": "OK"}
```

您也可以前往 Cloud Console 使用者介面，確認資料確實儲存在 Firestore 中：



從上方的螢幕截圖中，我們可以看到建立的 `books` 集合、以書籍 ISBN 碼識別的書籍文件清單，以及右側該特定書籍項目的詳細資料。

在雲端部署函式

如要在 Cloud Functions 中部署函式，請在 `function-import` 目錄中執行下列指令：

```
$ gcloud functions deploy bulk-import \
  --gen2 \
  --trigger-http \
  --runtime=nodejs20 \
  --allow-unauthenticated \
  --max-instances=30 \
  --region=${REGION} \
  --source=. \
  --entry-point=parseBooks
```

我們以 `bulk-import` 的符號名稱部署函式。這個函式是透過 HTTP 要求觸發，我們使用 Node.JS 20 執行階段。我們公開部署函式 (理想情況下，我們應保護該端點)。我們指定要讓函式所在的區域。我們指向本機目錄中的來源，並使用 `parseBooks` (匯出的 JavaScript 函式) 做為進入點。

幾分鐘後，函式就會部署到雲端。在 Cloud 控制台 UI 中，您應該會看到函式顯示：

Google Cloud Platform

serverless-web-apis

Search products and resources

Cloud Functions

Functions

CREATE FUNCTION

REFRESH

RELEASE NOTES

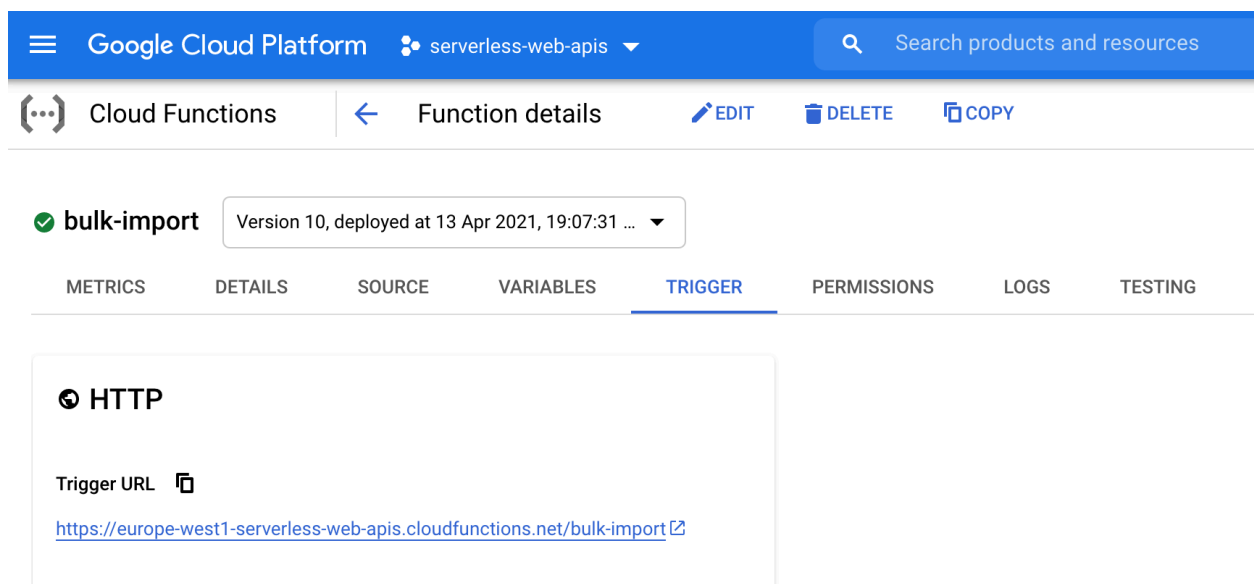
Filter

Filter functions

		Name	Region	Trigger	Runtime	Memory allocated	Executed function	Last deployed	Authentication	Actions
		bulk-import	europe-west3	HTTP	Node.js 12	256 MIB	parseBooks	11 Apr 2021, 16:30:07	Allow unauthenticated	

在部署輸出內容中，您應該可以看到函式的網址 (遵循特定命名慣例

`https://${REGION}-${GOOGLE_CLOUD_PROJECT}.cloudfunctions.net/${FUNCTION_NAME}`)，當然，您也可以 Cloud 控制台 UI 的「觸發條件」分頁中找到這個 HTTP 觸發條件網址：



Google Cloud Platform serverless-web-apis Search products and resources

Cloud Functions Function details EDIT DELETE COPY

bulk-import Version 10, deployed at 13 Apr 2021, 19:07:31 ...

METRICS DETAILS SOURCE VARIABLES TRIGGER PERMISSIONS LOGS TESTING

HTTP

Trigger URL

<https://europe-west1-serverless-web-apis.cloudfunctions.net/bulk-import>

您也可以透過指令列擷取網址：`gcloud`

```
$ export BULK_IMPORT_URL=$(gcloud functions describe bulk-import \
  --region=$REGION \
  --format 'value(httpsTrigger.url)')
$ echo $BULK_IMPORT_URL
```

讓我們將其儲存在 `BULK_IMPORT_URL` 環境變數中，以便重複使用，測試已部署的函式。

測試已部署的函式

使用與先前測試在本機執行的函式時類似的 `curl` 指令，測試已部署的函式。唯一變更的內容是網址：

```
$ curl -d "@../data/books.json" \  
      -H "Content-Type: application/json" \  
      $BULK_IMPORT_URL
```

同樣地，如果成功，應該會傳回下列輸出內容：

```
{"status": "OK"}
```

匯入函式已部署完畢，範例資料也已上傳，現在可以開始開發 REST API，公開這個資料集。

7. REST API 合約

雖然我們不會使用 [Open API 規範](#) 等方式定義 API 合約，但我們會查看 REST API 的各種端點。

API 會交換書籍 JSON 物件，其中包含：

- `isbn` (選填) - 代表有效 ISBN 碼的 13 個字元 `String`，
- `author` - 代表書籍作者名稱的非空白 `String`，
- `language` - 非空白的 `String`，內含書籍的撰寫語言，
- `pages`：書籍的頁數，必須是正數 `Integer`。
- `title` - 書名不得為空字串 `String`，
- `year`：書籍出版年份的 `Integer` 值。

書籍酬載範例：

```
{
  "isbn": "9780435272463",
  "author": "Chinua Achebe",
  "language": "English",
  "pages": 209,
  "title": "Things Fall Apart",
  "year": 1958
}
```

GET /books

取得所有書籍的清單，可依作者和/或語言篩選，並以每次 10 個結果的視窗分頁。

主體酬載：無。

查詢參數：

- `author` (選用) - 依作者篩選書籍清單，
- `language` (選用) - 依語言篩選書籍清單，
- `page` (選用，預設值為 0) - 表示要傳回的結果頁面排名。

傳回：書籍物件的 JSON 陣列。

狀態碼：

- `200`：要求成功擷取書籍清單時，
- `400` - 如果發生錯誤。

POST /books 和 POST /books/{isbn}

發布新的書籍酬載，可使用 `isbn` 路徑參數 (在這種情況下，書籍酬載中不需要 `isbn` 代碼)，或不使用該參數 (在這種情況下，書籍酬載中必須有 `isbn` 代碼)

主體酬載：書籍物件。

查詢參數：無。

傳回：無。

狀態碼：

- 201 - 書籍儲存成功時
- 406：如果 isbn 代碼無效，
- 400 - 如果發生錯誤。

GET /books/{isbn}

從圖書館中擷取書籍，並以路徑參數的形式傳遞書籍的 isbn 代碼。

主體酬載：無。

查詢參數：無。

傳回：書籍 JSON 物件，或書籍不存在時的錯誤物件。

狀態碼：

- 200：如果資料庫中找到該書籍，
- 400 - 如果發生錯誤，
- 404：如果找不到書籍，
- 406：如果 isbn 代碼無效。

PUT /books/{isbn}

更新現有書籍，並以做為路徑參數傳遞的 isbn 識別書籍。

主體酬載：書籍物件。您可以只傳遞需要更新的欄位，其他欄位則為選填。

查詢參數：無。

傳回：更新後的書籍。

狀態碼：

- 200：書籍更新成功時，
- 400 - 如果發生錯誤，
- 406：如果 isbn 代碼無效。

DELETE /books/{isbn}

刪除現有書籍，並以做為路徑參數傳遞的 isbn 識別。

主體酬載：無。

查詢參數：無。

傳回：無。

狀態碼：

- 204：書籍刪除成功時
 - 400 - 如果發生錯誤。
-

8. 在容器中部署及公開 REST API

探索程式碼

Dockerfile

首先，請查看 `Dockerfile`，這個檔案會負責將應用程式程式碼容器化：

```
FROM node:20-slim
WORKDIR /usr/src/app
COPY package*.json ./
RUN npm install --only=production
COPY . ./
CMD [ "node", "index.js" ]
```

我們使用的是 Node.JS 20「*slim*」映像檔。我們正在 `/usr/src/app` 目錄中作業。我們正在複製 `package.json` 檔案 (詳情請見下文)，該檔案定義了依附元件等項目。我們使用 `npm install` 安裝依附元件，並複製原始碼。最後，我們使用 `node index.js` 指令，指出應如何執行這個應用程式。

package.json

接著來看看 `package.json` 檔案：

```
{
  "name": "run-crud",
  "description": "CRUD operations over book data",
  "license": "Apache-2.0",
  "engines": {
    "node": ">= 20.0.0"
  },
  "dependencies": {
    "@google-cloud/firestore": "^4.9.9",
    "cors": "^2.8.5",
    "express": "^4.17.1",
    "isbn3": "^1.1.10"
  },
  "scripts": {
    "start": "node index.js"
  }
}
```

我們指定要使用 Node.JS 14，這與 `Dockerfile` 的情況相同。

我們的 Web API 應用程式取決於：

- Firestore NPM 模組，用於存取資料庫中的書籍資料。
- `cors` 程式庫，用於處理 [CORS](#) (跨來源資源共用) 要求，因為我們的 REST API 會從 App Engine 網頁應用程式前端的用戶端程式碼叫用，
- Express 架構，這是我們設計 API 時使用的網路架構。
- 接著是 `isbn3` 模組，可協助驗證書籍 ISBN 碼。

我們也指定了 `start` 指令碼，方便您在本地啟動應用程式，以進行開發和測試。

index.js

接著深入瞭解 `index.js`，看看程式碼的重點：

```
const Firestore = require('@google-cloud/firestore');
const firestore = new Firestore();
const bookStore = firestore.collection('books');
```

我們需要 Firestore 模組，並參照儲存書籍資料的 `books` 集合。

```
const express = require('express');
const app = express();
const bodyParser = require('body-parser');
app.use(bodyParser.json());

const querystring = require('querystring');

const cors = require('cors');
app.use(cors({
  exposedHeaders: ['Content-Length', 'Content-Type', 'Link'],
}));
```

我們使用 Express 做為網路架構，實作 REST API。我們使用 `body-parser` 模組剖析與 API 交換的 JSON 酬載。

`querystring` 模組有助於操控網址。當我們為分頁建立 `Link` 標頭時，就會發生這種情況 (稍後會詳細說明)。

接著設定 `cors` 模組。我們明確指定要透過 CORS 傳遞的標頭，因為通常大多數標頭都會遭到移除，但在此我們希望保留一般內容長度和類型，以及為分頁指定的 `Link` 標頭。

```
const ISBN = require('isbn3');

function isbnOK(isbn, res) {
  const parsedIsbn = ISBN.parse(isbn);
  if (!parsedIsbn) {
    res.status(406)
      .send({error: `Invalid ISBN: ${isbn}`});
    return false;
  }
  return parsedIsbn;
}
```

我們將使用 `isbn3` NPM 模組剖析及驗證 ISBN 代碼，並開發小型公用函式來剖析 ISBN 代碼，如果 ISBN 代碼無效，則會在回應中傳回 `406` 狀態碼。

- **GET /books**

讓我們逐一查看 `GET /books` 端點：

```

app.get('/books', async (req, res) => {
  try {
    var query = new Firestore().collection('books');

    if (!!req.query.author) {
      console.log(`Filtering by author: ${req.query.author}`);
      query = query.where("author", "=", req.query.author);
    }
    if (!!req.query.language) {
      console.log(`Filtering by language: ${req.query.language}`);
      query = query.where("language", "=", req.query.language);
    }

    const page = parseInt(req.query.page) || 0;

    // - - ✂ - - ✂ - - ✂ - - ✂ - -

  } catch (e) {
    console.error('Failed to fetch books', e);
    res.status(400)
      .send({error: `Impossible to fetch books: ${e.message}`});
  }
});

```

我們即將準備查詢資料庫，方法是準備查詢。這項查詢會視選用的查詢參數而定，可依作者和/或語言篩選。我們也會以 10 本為單位傳回書籍清單。

如果在擷取書籍時發生錯誤，我們會傳回 400 狀態碼的錯誤。

讓我們放大該端點的剪除部分：

```

const snapshot = await query
  .orderBy('updated', 'desc')
  .limit(PAGE_SIZE)
  .offset(PAGE_SIZE * page)
  .get();

const books = [];

if (snapshot.empty) {
  console.log('No book found');
} else {
  snapshot.forEach(doc => {
    const {title, author, pages, year, language, ...otherFields} = doc.data();
    const book = {isbn: doc.id, title, author, pages, year, language};
    books.push(book);
  });
}

```

在前一節中，我們依 `author` 和 `language` 進行篩選，但本節要依上次更新日期排序書籍清單（上次更新的書籍排在最前面）。我們也會定義限制（要傳回的元素數量）和偏移量（要從哪個起點傳回下一批書籍），將結果分頁。

注意：分頁功能可能比您想像的更複雜，尤其是使用 Firestore 這類 NoSQL 文件導向資料庫時。使用 `limit / offset` 的缺點是，Firestore 一律會從頭到 `offset+limit` 讀取所有資料，而不只是讀取偏移量和 `offset+limit` 之間的資料。如果需要處理大量資料，這可能需要花費大量時間和金錢。

建議您參閱[說明文件](#)，並觀看這部詳細的[影片](#)，瞭解使用 Firestore 進行分頁的不同方法。

我們執行查詢、取得資料的快照，並將這些結果放入 JavaScript 陣列，最後在函式結尾傳回。

讓我們看看最佳做法，完成這個端點的說明：使用 `Link` 標頭定義資料第一頁、前一頁、下一頁或最後一頁的 URI 連結 (在本例中，我們只會提供前一頁和下一頁)。

```
var links = {};  
if (page > 0) {  
  const prevQuery = querystring.stringify({...req.query, page: page - 1});  
  links.prev = `${req.path}${prevQuery != '' ? `?${prevQuery}` : ''}`;  
}  
if (snapshot.docs.length === PAGE_SIZE) {  
  const nextQuery = querystring.stringify({...req.query, page: page + 1});  
  links.next = `${req.path}${nextQuery != '' ? `?${nextQuery}` : ''}`;  
}  
if (Object.keys(links).length > 0) {  
  res.links(links);  
}  
  
res.status(200).send(books);
```

一開始，這個邏輯可能有點複雜，但我們所做的，是如果不在資料的第一頁，就新增「上一頁」連結。如果資料頁面已滿 (即包含 `PAGE_SIZE` 常數定義的書籍數量上限，且假設還有其他頁面包含更多資料)，我們會新增 `next` 連結。接著，我們會使用 Express 的 `resource#links()` 函式，以正確語法建立正確的標頭。

供您參考，連結標頭如下所示：

```
link: </books?page=1>; rel="prev", </books?page=3>; rel="next"
```

- `POST /books` 和 `POST /books/:isbn`

這兩個端點都是用來建立新書。其中一個是在書籍酬載中傳遞 ISBN 碼，另一個則是將 ISBN 碼當做路徑參數傳遞。無論是哪種方式，都會呼叫 `createBook()` 函式：

```

async function createBook(isbn, req, res) {
  const parsedIsbn = isbnOK(isbn, res);
  if (!parsedIsbn) return;

  const {title, author, pages, year, language} = req.body;

  try {
    const docRef = bookStore.doc(parsedIsbn.isbn13);
    await docRef.set({
      title, author, pages, year, language,
      updated: Firestore.Timestamp.now()
    });
    console.log(`Saved book ${parsedIsbn.isbn13}`);

    res.status(201)
      .location(`/books/${parsedIsbn.isbn13}`)
      .send({status: `Book ${parsedIsbn.isbn13} created`});
  } catch (e) {
    console.error(`Failed to save book ${parsedIsbn.isbn13}`, e);
    res.status(400)
      .send({error: `Impossible to create book ${parsedIsbn.isbn13}: ${e.message}`});
  }
}

```

我們會檢查 `isbn` 程式碼是否有效，否則會從函式傳回 (並設定 `406` 狀態碼)。我們會從要求主體中傳遞的酬載擷取書籍欄位。接著，我們會將書籍詳細資料儲存在 Firestore 中。成功時傳回 `201`，失敗時傳回 `400`。

成功傳回時，我們也會設定位置標頭，向 API 用戶端提供新建立資源的位置提示。標頭如下所示：

```
Location: /books/9781234567898
```

- **GET /books/:isbn**

現在從 Firestore 擷取書籍，並透過 ISBN 識別。

```

app.get('/books/:isbn', async (req, res) => {
  const parsedIsbn = isbnOK(req.params.isbn, res);
  if (!parsedIsbn) return;

  try {
    const docRef = bookStore.doc(parsedIsbn.isbn13);
    const docSnapshot = await docRef.get();

    if (!docSnapshot.exists) {
      console.log(`Book not found ${parsedIsbn.isbn13}`)
      res.status(404)
        .send({error: `Could not find book ${parsedIsbn.isbn13}`});
      return;
    }

    console.log(`Fetched book ${parsedIsbn.isbn13}`, docSnapshot.data());

    const {title, author, pages, year, language, ...otherFields} = docSnapshot.data();
    const book = {isbn: parsedIsbn.isbn13, title, author, pages, year, language};

    res.status(200).send(book);
  } catch (e) {
    console.error(`Failed to fetch book ${parsedIsbn.isbn13}`, e);
    res.status(400)
      .send({error: `Impossible to fetch book ${parsedIsbn.isbn13}: ${e.message}`});
  }
});

```

我們會照常檢查 ISBN 是否有效。我們向 Firestore 查詢，以擷取書籍。如果確實找到書籍，`snapshot.exists` 屬性就很有用。否則，我們會傳回錯誤和 404 「找不到」狀態碼。我們會擷取書籍資料，並建立代表書籍的 JSON 物件，以便傳回。

- **PUT /books/:isbn**

我們使用 PUT 方法更新現有書籍。

```

app.put('/books/:isbn', async (req, res) => {
  const parsedIsbn = isbnOK(req.params.isbn, res);
  if (!parsedIsbn) return;

  try {
    const docRef = bookStore.doc(parsedIsbn.isbn13);
    await docRef.set({
      ...req.body,
      updated: Firestore.Timestamp.now()
    }, {merge: true});
    console.log(`Updated book ${parsedIsbn.isbn13}`);

    res.status(201)
      .location(`/books/${parsedIsbn.isbn13}`)
      .send({status: `Book ${parsedIsbn.isbn13} updated`});
  } catch (e) {
    console.error(`Failed to update book ${parsedIsbn.isbn13}`, e);
    res.status(400)
      .send({error: `Impossible to update book ${parsedIsbn.isbn13}: ${e.message}`});
  }
});

```

我們會更新 `updated` 日期/時間欄位，記錄上次更新該記錄的時間。我們採用 `{merge:true}` 策略，將現有欄位替換為新值 (否則所有欄位都會遭到移除，只有酬載中的新欄位會儲存，從而清除先前更新或初始建立時的現有欄位)。

我們也設定了 `Location` 標頭，指向書籍的 URI。

- **DELETE /books/:isbn**

刪除書籍的步驟相當簡單。我們只需在文件參照上呼叫 `delete()` 方法。由於我們不會傳回任何內容，因此會傳回 204 狀態碼。

```

app.delete('/books/:isbn', async (req, res) => {
  const parsedIsbn = isbnOK(req.params.isbn, res);
  if (!parsedIsbn) return;

  try {
    const docRef = bookStore.doc(parsedIsbn.isbn13);
    await docRef.delete();
    console.log(`Book ${parsedIsbn.isbn13} was deleted`);

    res.status(204).end();
  } catch (e) {
    console.error(`Failed to delete book ${parsedIsbn.isbn13}`, e);
    res.status(400)
      .send({error: `Impossible to delete book ${parsedIsbn.isbn13}: ${e.message}`});
  }
});

```

啟動 Express / Node 伺服器

最後，我們啟動伺服器，預設會監聽通訊埠 `8080`：


```
const port = process.env.PORT || 8080;
app.listen(port, () => {
  console.log(`Books Web API service: listening on port ${port}`);
  console.log(`Node ${process.version}`);
});
```

在本機執行應用程式

如要在本機執行應用程式，請先使用下列指令安裝依附元件：

```
$ npm install
```

然後從以下內容開始：

```
$ npm start
```

伺服器會在 `localhost` 上啟動，並預設監聽通訊埠 8080。

您也可以使用下列指令建構 Docker 容器，並執行容器映像檔：

```
$ docker build -t crud-web-api .

$ docker run --rm -p 8080:8080 -it crud-web-api
```

在 Docker 中執行也是檢查應用程式容器化作業是否能順利運作的好方法，因為我們會在 Cloud Build 中建構應用程式。

測試 API

無論我們如何執行 REST API 程式碼 (直接透過 Node 或 Docker 容器映像檔)，現在都能對其執行幾項查詢。

- 建立新書 (主體酬載中的 ISBN)：

```
$ curl -XPOST -d
'{"isbn":"9782070368228","title":"Book","author":"me","pages":123,"year":2021,"language":"French"}' \
-H "Content-Type: application/json" \
http://localhost:8080/books
```

- 建立新書 (路徑參數中的 ISBN)：

```
$ curl -XPOST -d
'{"title":"Book","author":"me","pages":123,"year":2021,"language":"French"}' \
-H "Content-Type: application/json" \
http://localhost:8080/books/9782070368228
```

- 刪除書籍 (我們建立的書籍)：

```
$ curl -XDELETE http://localhost:8080/books/9782070368228
```

- 依 ISBN 擷取書籍：

```
$ curl http://localhost:8080/books/9780140449136
$ curl http://localhost:8080/books/9782070360536
```

- 如要更新現有書籍，只須變更書名：

```
$ curl -XPUT \
  -d '{"title":"Book"}' \
  -H "Content-Type: application/json" \
  http://localhost:8080/books/9780003701203
```

- 擷取書籍清單 (前 10 本)：

```
$ curl http://localhost:8080/books
```

- 尋找特定作者撰寫的書籍：

```
$ curl http://localhost:8080/books?author=Virginia+Woolf
```

- 列出以英文撰寫的書籍：

```
$ curl http://localhost:8080/books?language=English
```

- 載入第 4 頁的書籍：

```
$ curl http://localhost:8080/books?page=3
```

我們也可以合併 `author`、`language` 和 `books` 查詢參數，進一步修正搜尋結果。

建構及部署容器化 REST API

我們很高興 REST API 運作正常，現在正是將其部署到 Cloud Run 的好時機！

我們將分兩個步驟完成這項作業：

- 首先，使用 [Cloud Build](#) 建構容器映像檔，方法是執行下列指令：

```
$ gcloud builds submit \
  --tag gcr.io/${GOOGLE_CLOUD_PROJECT}/crud-web-api
```

- 然後使用第二個指令部署服務：

```
$ gcloud run deploy run-crud \
  --image gcr.io/${GOOGLE_CLOUD_PROJECT}/crud-web-api \
  --allow-unauthenticated \
  --region=${REGION} \
  --platform=managed
```

透過第一個指令，Cloud Build 會建構容器映像檔，並將其代管於 Container Registry。下一個指令會從登錄檔部署容器映像檔，並部署至雲端區域。

我們可以在 Cloud 控制台 UI 中再次確認，Cloud Run 服務現在會顯示在清單中：

Google Cloud Platform

serverless-web-api

Search products and resources

Cloud Run

Services

CREATE SERVICE

MANAGE CUSTOM DOMAINS

COPY

DELETE

SHOW INFO PANEL

Each Cloud Run service has a unique endpoint and auto-scales deployed containers.

Learn more

Filter

Filter services

☐ ☒ Name ↑	Req/sec	Region	Authentication	Ingress	Last deployed	Deployed by
☐ ☒ run-crud	0	europe-west1	Allow unauthenticated	All	12 Apr 2021, 13:54:44	glademo@cloudadvocacyorg.joonix.net

最後一個步驟是使用下列指令，擷取新部署的 Cloud Run 服務網址：

```
$ export RUN_CRUD_SERVICE_URL=$(gcloud run services describe run-crud \
  --region=${REGION} \
  --platform=managed \
  --format='value(status.url)')
```

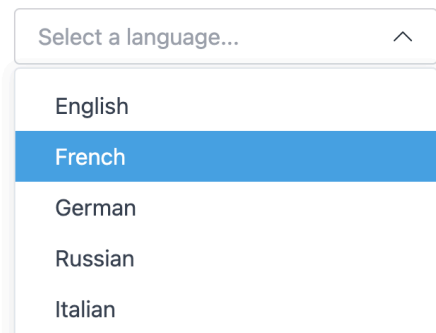
在下一個部分中，我們需要 Cloud Run REST API 的網址，因為 App Engine 前端程式碼會與該 API 互動。

9. 託管網頁應用程式，瀏覽程式庫

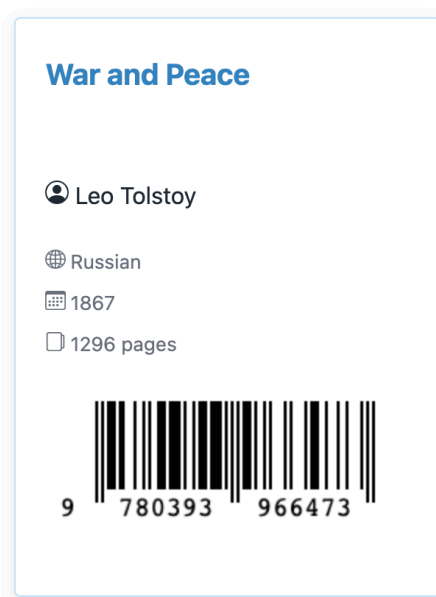
最後一塊拼圖是提供網頁前端，與 REST API 互動，為這個專案增添光彩。為此，我們將使用 Google App Engine，搭配一些用戶端 JavaScript 程式碼，透過 AJAX 要求呼叫 API (使用用戶端 [Fetch API](#))。

我們的應用程式雖然部署在 Node.js App Engine 執行階段，但大部分都是靜態資源！由於大部分使用者互動都會透過用戶端 JavaScript 在瀏覽器中進行，因此後端程式碼不多。我們不會使用任何花俏的前端 JavaScript 架構，只會使用一些「原生」JavaScript，並透過 [Shoelace](#) 網頁元件程式庫，為 UI 建立幾個網頁元件：

- 選取方塊，選擇書籍語言：



- 顯示特定書籍詳細資料的資訊卡元件 (包括代表書籍 ISBN 的條碼，使用 [JsBarcode](#) 程式庫)：



- 以及從資料庫載入更多書籍的按鈕：



將所有這些視覺元件組合在一起後，瀏覽程式庫的網頁會如下所示：

Book library

French



Memoirs of Hadrian

 Marguerite Yourcenar

 French

 1951

 408 pages



Gargantua and Pantagruel

 François Rabelais

 French

 1533

 623 pages



The Red and the Black

 Stendhal

 French

 1830

 576 pages



`app.yaml`

設定檔

首先，請查看這個 App Engine 應用程式的 `app.yaml` 設定檔，深入瞭解程式碼集。這是 App Engine 專用的檔案，可設定環境變數、應用程式的各種「處理常式」，或指定某些資源為靜態資產，由 App Engine 內建的 CDN 提供服務。

```
runtime: nodejs14

env_variables:
  RUN_CRUD_SERVICE_URL: CHANGE_ME

handlers:

- url: /js
  static_dir: public/js

- url: /css
  static_dir: public/css

- url: /img
  static_dir: public/img

- url: /(.+\.html)
  static_files: public/html/\1
  upload: public/(.+\.html)

- url: /
  static_files: public/html/index.html
  upload: public/html/index\*.html

- url: /*
  secure: always
  script: auto
```

我們指定應用程式為 Node.JS，並使用版本 14。

接著，我們定義指向 Cloud Run 服務網址的環境變數。我們需要使用正確的網址更新 CHANGE_ME 預留位置 (請參閱下文，瞭解如何變更)。

接著定義各種處理常式。前 3 個指向 public/ 資料夾及其子資料夾下的 HTML、CSS 和 JavaScript 用戶端程式碼位置。第四個表示 App Engine 應用程式的根網址應指向 index.html 頁面。這樣一來，我們在存取網站根目錄時，就不會在網址中看到 index.html 尾碼。最後一個是預設值，會將所有其他網址 (/.*) 轉送至 Node.JS 應用程式 (也就是應用程式的「動態」部分，與我們說明的靜態資產相對)。

現在來更新 Cloud Run 服務的 Web API 網址。

在 appengine-frontend/ 目錄中執行下列指令，更新指向 Cloud Run 型 REST API 網址的環境變數：

```
$ sed -i -e "s|CHANGE_ME|${RUN_CRUD_SERVICE_URL}|" app.yaml
```

或者，在 app.yaml 中手動將 CHANGE_ME 字串變更為正確的網址：

```
env_variables:
  RUN_CRUD_SERVICE_URL: CHANGE_ME
```

Node.JS package.json 檔案

```
{
  "name": "appengine-frontend",
  "description": "Web frontend",
  "license": "Apache-2.0",
  "main": "index.js",
  "engines": {
    "node": "^14.0.0"
  },
  "dependencies": {
    "express": "^4.17.1",
    "isbn3": "^1.1.10"
  },
  "devDependencies": {
    "nodemon": "^2.0.7"
  },
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon --watch server --inspect index.js"
  }
}
```

我們再次強調，我們想使用 Node.JS 14 執行這個應用程式。我們依附於 Express 框架，以及用於驗證書籍 ISBN 碼的 `isbn3` NPM 模組。

在開發依附元件中，我們將使用 `nodemon` 模組監控檔案變更。雖然我們可以透過 `npm start` 在本機執行應用程式、變更程式碼、使用 `^c` 停止應用程式，然後重新啟動，但這樣做有點麻煩。我們可以改用下列指令，讓應用程式在變更時自動重新載入 / 重新啟動：

```
$ npm run dev
```

`index.js` Node.JS 程式碼

```
const express = require('express');
const app = express();

app.use(express.static('public'));

const bodyParser = require('body-parser');
app.use(bodyParser.json());
```

我們需要 Express 網路框架。我們指定公開目錄包含可由 `static` 中介軟體提供的靜態資產 (至少在本機以開發模式執行時)。最後，我們需要 `body-parser` 剖析 JSON 酬載。

讓我們看看定義的幾條路徑：

```
app.get('/', async (req, res) => {
  res.redirect('/html/index.html');
});

app.get('/webapi', async (req, res) => {
  res.send(process.env.RUN_CRUD_SERVICE_URL);
});
```

第一個符合 `/` 的項目會重新導向至 `public/html` 目錄中的 `index.html`。由於我們並未在開發模式下執行 App Engine 執行階段，因此不會進行 App Engine 的網址路由作業。因此，我們在這裡只是將根網址重新導向至 HTML 檔案。

我們定義的第二個端點 `/webapi` 會傳回 Cloud Run REST API 的網址。這樣一來，用戶端 JavaScript 程式碼就會知道要呼叫哪個位置來取得書籍清單。

```
const port = process.env.PORT || 8080;
app.listen(port, () => {
  console.log(`Book library web frontend: listening on port ${port}`);
  console.log(`Node ${process.version}`);
  console.log(`Web API endpoint ${process.env.RUN_CRUD_SERVICE_URL}`);
});
```

最後，我們執行 Express 網頁應用程式，並預設監聽通訊埠 8080。

`index.html` 頁面

我們不會查看這個長 HTML 網頁的每一行，我們改為醒目顯示幾行重要程式碼。

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@shoelace-style/shoelace@2.0.0-beta.37/dist/themes/base.css">
<script type="module" src="https://cdn.jsdelivr.net/npm/@shoelace-style/shoelace@2.0.0-beta.37/dist/shoelace.js"></script>

<script src="https://cdn.jsdelivr.net/npm/jsbarcode@3.11.0/dist/barcodes/JsBarcode.ean-upc.min.js"></script>

<script src="/js/app.js"></script>
<link rel="stylesheet" type="text/css" href="/css/style.css">
```

前兩行會匯入 Shoelace 網頁元件程式庫 (指令碼和樣式表)。

下一行會匯入 JsBarcode 程式庫，建立書籍 ISBN 碼的條碼。

最後幾行會匯入我們自己的 JavaScript 程式碼和 CSS 樣式表，這些程式碼和樣式表位於 `public/` 子目錄中。

在 HTML 網頁的 `body` 中，我們使用 Shoelace 元件及其自訂元素標記，例如：


```

<sl-icon name="book-half"></sl-icon>
...

<sl-select id="language-select" placeholder="Select a language..." clearable>
  <sl-menu-item value="English">English</sl-menu-item>
  <sl-menu-item value="French">French</sl-menu-item>
  ...
</sl-select>
...

<sl-button id="more-button" type="primary" size="large">
  More books...
</sl-button>
...

```

我們也會使用 HTML 範本及其填寫功能來呈現書籍。我們會建立該範本的副本，填入書籍清單，並將預留位置中的值替換為書籍詳細資料：

```

<template id="book-card">
  <sl-card class="card-overview">
    ...
    <slot name="author">Author</slot>
    ...
  </sl-card>
</template>

```

HTML 講得差不多了，我們即將完成程式碼審查。最後一個重要部分：與 REST API 互動的 `app.js` 用戶端 JavaScript 程式碼。

app.js 用戶端 JavaScript 程式碼

首先，我們從等待 DOM 內容載入的頂層事件監聽器開始：

```

document.addEventListener("DOMContentLoaded", async function(event) {
  ...
})

```

準備就緒後，我們就可以設定一些重要常數和變數：

```

const serverUrlResponse = await fetch('/webapi');
const serverUrl = await serverUrlResponse.text();
console.log('Web API endpoint:', serverUrl);

const server = serverUrl + '/books';
var page = 0;
var language = '';

```

首先，我們會擷取 REST API 的網址，這要歸功於 App Engine 節點程式碼，該程式碼會傳回我們最初在 `app.yaml` 中設定的環境變數。由於環境變數的關係，從 JavaScript 用戶端程式碼呼叫的 `/webapi` 端點，不必在前端程式碼中硬式編碼

REST API 網址。

我們也會定義 `page` 和 `language` 變數，用於追蹤分頁和語言篩選。

```
const moreButton = document.getElementById('more-button');
moreButton.addEventListener('sl-focus', event => {
  console.log('Button clicked');
  moreButton.blur();

  appendMoreBooks(server, page++, language);
});
```

我們會在按鈕上新增事件處理常式，以便載入書籍。點選時，系統會呼叫 `appendMoreBooks()` 函式。

```
const langSelect = document.getElementById('language-select');
langSelect.addEventListener('sl-change', event => {
  page = 0;
  language = event.srcElement.value;
  document.getElementById('library').replaceChildren();
  console.log(`Language selected: "${language}"`);

  appendMoreBooks(server, page++, language);
});
```

選取方塊也一樣，我們新增了事件處理常式，以便在語言選取項目變更時收到通知。與按鈕一樣，我們也會呼叫 `appendMoreBooks()` 函式，傳遞 REST API 網址、目前網頁和語言選項。

現在來看看擷取及附加書籍的函式：

```
async function appendMoreBooks(server, page, language) {
  const searchUrl = new URL(server);
  if (!!page) searchUrl.searchParams.append('page', page);
  if (!!language) searchUrl.searchParams.append('language', language);

  const response = await fetch(searchUrl.href);
  const books = await response.json();
  ...
}
```

如上所示，我們正在製作要用於呼叫 REST API 的確切網址。我們通常可以指定三個查詢參數，但在此 UI 中，我們只指定兩個：

- `page`：整數，表示書籍分頁的目前頁面。
- `language`：用於依書寫語言篩選的語言字串。

接著，我們使用 Fetch API 擷取包含書籍詳細資料的 JSON 陣列。

```
const linkHeader = response.headers.get('Link')
console.log('Link', linkHeader);
if (!!linkHeader && linkHeader.indexOf('rel="next"') > -1) {
  console.log('Show more button');
  document.getElementById('buttons').style.display = 'block';
} else {
  console.log('Hide more button');
  document.getElementById('buttons').style.display = 'none';
}
```

我們會根據回應中是否有 `Link` 標頭，顯示或隱藏 `[More books...]` 按鈕，因為 `Link` 標頭會提示我們是否還有更多書籍要載入 (`Link` 標頭中會有 `next` 網址)。

```
const library = document.getElementById('library');
const template = document.getElementById('book-card');
for (let book of books) {
  const bookCard = template.content.cloneNode(true);

  bookCard.querySelector('slot[name=title]').innerText = book.title;
  bookCard.querySelector('slot[name=language]').innerText = book.language;
  bookCard.querySelector('slot[name=author]').innerText = book.author;
  bookCard.querySelector('slot[name=year]').innerText = book.year;
  bookCard.querySelector('slot[name=pages]').innerText = book.pages;

  const img = document.createElement('img');
  img.setAttribute('id', book.isbn);
  img.setAttribute('class', 'img-barcode-' + book.isbn);
  bookCard.querySelector('slot[name=barcode]').appendChild(img);

  library.appendChild(bookCard);
  ...
}
}
```

在上述函式部分中，針對 REST API 傳回的每本書，我們將複製範本 (其中包含代表書籍的某些網頁元件)，並在範本的 `slot` 中填入書籍詳細資料。

```
JsBarcode('.img-barcode-' + book.isbn).EAN13(book.isbn, {fontSize: 18, textMargin: 0, height: 60}).render();
```

為了讓 ISBN 碼更美觀，我們使用 `JsBarcode` 程式庫建立精美的條碼，就像實體書封底的條碼一樣！

在本機執行及測試應用程式

目前程式碼已足夠，接下來讓我們實際操作應用程式。首先，我們會在 `Cloud Shell` 中進行這項操作，然後再實際部署。

我們使用下列指令安裝應用程式所需的 `NPM` 模組：

```
$ npm install
```

然後我們可以使用一般方式執行應用程式：

```
$ npm start
```

或使用 `nodemon` 自動重新載入變更，方法如下：

```
$ npm run dev
```

應用程式會在本地執行，我們可以在瀏覽器中透過 <http://localhost:8080> 存取應用程式。

部署 App Engine 應用程式

現在我們確信應用程式在本機上運作正常，因此可以將其部署至 App Engine。

如要部署應用程式，請啟動下列指令：

```
$ gcloud app deploy -q
```

大約一分鐘後，應用程式就會部署完成。

應用程式網址的格式為：`https://${GOOGLE_CLOUD_PROJECT}.appspot.com`。

探索 App Engine 網頁應用程式的 UI

您現在可以：

- 按一下 `[More books...]` 按鈕即可載入更多書籍。
 - 選取特定語言，即可只查看該語言的書籍。
 - 如要清除選取項目並返回所有書籍的清單，請按一下選取方塊中的小十字。
-

步驟 10 · 約 1 分鐘

10. 清理 (選用)

如果您不打算保留應用程式，可以刪除整個專案來清理資源，節省費用，並當個優質的雲端使用者：

```
gcloud projects delete ${GOOGLE_CLOUD_PROJECT}
```

11. 恭喜！

我們運用 Cloud Functions、App Engine 和 Cloud Run 建立了一組服務，公開各種 Web API 端點和網頁前端，並遵循 REST API 開發作業的良好設計模式，儲存、更新及瀏覽書籍庫。

涵蓋內容

- Cloud Functions
- Cloud Firestore
- Cloud Run
- App Engine

進階做法

如要進一步探索及擴展這個具體範例，請參閱下列清單：

- 善用 [API Gateway](#)，為資料匯入函式和 REST API 容器提供常見的 API 外觀，新增處理 API 金鑰以存取 API 等功能，或為 API 消費者定義速率限制。
 - 在 App Engine 應用程式中部署 [Swagger-UI](#) 節點模組，記錄 REST API 並提供測試遊樂場。
 - 在前端，除了現有的瀏覽功能外，還可新增額外畫面來編輯資料、建立新的書籍項目。此外，由於我們使用 Cloud Firestore 資料庫，因此可以運用其[即時](#)功能，在變更時更新顯示的書籍資料。
-